Formalizing the Game of Skull in Alloy

Iris Chen

System Overview: The Game of *Skull*

Skull is a bluffing card game typically played between two or more players. This Alloy model considers a two-player version with PlayerA and PlayerB. Each player starts with four cards: three *Flowers* and one *Skull*. The game proceeds through four phases: **Placing**, **Bidding**, **Flipping**, and **Winning**. The first player to win two points is declared the winner.

The model captures key elements of gameplay, including card placements, bidding logic, card flipping, and win/loss conditions.

Alloy Model Explanation

Signatures

- `Player`: Abstract type extended by `PlayerA` and `PlayerB`.
- `Card`: Abstract type with one `Skull` and three `Flower` cards.
- `Phase`: Game phases: `Placing`, `Bidding`, `Flipping`, and `Winning`.
- `State`: Represents a snapshot of the game. Includes fields for players' hands, card stacks, flip zones, current phase, turn, bidding status, and scores.

Predicates and Functions

`Init` Initializes the game state: all cards are in hand, no stack or flip cards, bidding is inactive, phase is `Placing`, and PlayerA starts.

`otherPlayer` A helper function returning the opponent of a given player.

`PlaceCard` Models a player placing one or more cards from their hand to the stack. `stackA` and `stackB` keep track of the order in which players place their cards. Turn switches to the other player. If a player reaches score 2, the game enters the `Winning` phase.

`PassPlace` Allows a player to pass instead of placing a card. Passing is only allowed once both players have placed at least one card, or if a player has no cards left. Transitions to `Bidding` phase.

`Bid` Represents a player making a bid during the `Bidding` phase. If no one has bid yet (bid = 0), then any player can start bidding with a value between 1 and the total number of cards placed (stackA and stackB). If a bid has already been made, then the new bid must be strictly greater than the previous one but not exceed the total placed cards. The player who bids becomes the new bidder. Turn switches accordingly.

`PassBid` Models a player passing instead of raising the bid. If the bid reaches the total number of placed cards, the player whose turn it is not automatically passes. Once passed, the game moves to `Flipping`, where the bidder begins flipping cards.

`Flip` Handles the sequential flipping of cards: starting with the last card placed on the player's own stack, and continuing to the opponent's stack if more cards need to be flipped. When cards are flipped, they're added to either flipA or flipB in order depending on which stack they came from—stackA or stackB. At the same time, the flipped cards are removed from their original stacks. Flipping continues until the bid is fulfilled or a `Skull` is revealed.

`LoseCardA / LoseCardB` If a `Skull` is revealed, the player who flipped it loses one card randomly from their pool (hand, flipped cards, and stack). Another player's full set of cards (stack + flip + hand) is returned to them, and a new round begins with the player who is not the bidder.

`WinA / WinB` If the bidder flips the number of cards they bid without revealing a `Skull`, they win the round and gain a point. If their score reaches 2, the game enters a terminal `Winning` state. Otherwise, a new round begins with the same player.

`Finish` Captures a final game state with no actions left. Ensures consistency of score and card ownership going forward.

`Game` The main game predicate chaining all possible transitions: from `Init` through all valid state changes in `Placing`, `Bidding`, `Flipping`, and finally to `Winning`.

Analysis

The first question explored using the Alloy model is: *What do the minimal solutions look like?* The smallest valid trace of the game, as returned by Alloy, consists of 15 states. In these minimal solutions, both PlayerA and PlayerB place only one card in their respective stacks during the placing phase, and the highest bid made is one. Because PlayerA always starts the game, and the next round also begins with the same player if they win, PlayerA consistently wins both rounds in the shortest traces. These traces do not involve any complex bluffing or card loss scenarios—no player places more than one card, no Skulls are flipped, and no players lose cards. This confirms that the minimal solution reflects a "clean win" scenario with minimal interaction and maximum symmetry.

The second question asks: *If a player loses their Skull, does that guarantee their eventual loss?* The answer, based on Alloy's counterexamples, is no. While losing a Skull can reduce a player's bluffing capacity and increase their vulnerability, it does not ensure defeat. Alloy-generated traces demonstrate scenarios in which PlayerA loses their Skull early in the game but still manages to win in three rounds. These traces illustrate that success in Skull depends more on placing cards, bidding decisions, and card flipping outcomes than on simply retaining all four cards. In fact, strategic play can still compensate for the loss of a Skull, particularly if the remaining Flowers are played carefully and the opponent miscalculates their bid or flip.

The third question asks: *In minimal solutions, is it possible for a player who has lost their Skull to still win?* Upon investigation, the answer is no. In the minimal traces found by Alloy, no player ever loses a card, meaning both players always retain their original hands throughout the entire game. As a result, the condition `Skull not in s.A` or `Skull not in s.B` is never satisfied in a `Winning` state. Because the assertion concerning Skull loss is written as an implication, and the premise never holds in these minimal cases, Alloy considers the assertion vacuously true. Thus, Alloy does not refute it, even though more complex traces clearly allow players to win after losing their Skull.

Thoughts

I think the part that tripped me up the most was the flipping phase. At first, I tried to flip all the cards at once, which required calculating a subsequence that contained the correct elements (cards). I went through several versions of this approach, but the results were never what I expected. The issue ended up being in the calculation, which was hard to catch just by looking at the output. Eventually, I realized it's better to

model state transitions one minimal move at a time. So I rewrote the `Flip` predicate to use Alloy's built-in sequence helper functions directly, which made the logic much clearer than my earlier versions. From there, I could model how the game transitions from flipping to different outcomes depending on the situation.

Another general challenge was managing the many conditions in different phases. I mostly relied on visualization to add the right constraints and debug the model. Also, early on, the state transitions during the flipping phase didn't behave as expected. No matter how much I increased the scope, Alloy wouldn't show the `LoseCardA` or `LoseCardB` states. In the end, I fixed it by moving the relevant constraints into the `LoseCardA`, `LoseCardB`, `WinA`, and `WinB` predicates themselves, instead of putting everything in the main logic of the game.

One direction for future work is modeling different player strategies to better simulate realistic gameplay behavior. Currently, the Alloy model allows all possible transitions, but does not capture intentional or tactical decision-making. A natural extension would be to define predicates that encode specific strategies, such as risk-averse players who avoid bidding high if they placed a Skull, or aggressive players who always open with strong bids when they know they played a Flower. Other strategies might include passing when not the current highest bidder, or mimicking an opponent's last action. These strategic behaviors could be encoded as optional constraints layered on top of the existing transition predicates. By associating each game trace with a strategy label and tracking outcomes (e.g. final scores), the model could be used to explore which styles of play are more successful in different contexts.

Another important direction is generalizing the model to support more than two players. Skull is typically played with 3–6 participants, so capturing the full dynamics of the game would require restructuring many parts of the model. This includes replacing fixed fields like `stackA` and `stackB` with mappings from players to their stacks, and defining a rotation mechanism for player turns. Bidding and passing logic would also need to handle arbitrary sequences of players, introducing more complexity in tracking active participants and determining winners.